

Lecture 6

FAIR in Practice

Interoperability · Reusability · Reproducibility



Guest contribution: Jamie McCann, PlantGenIE developer, Umeå University

Today you will query real research infrastructure from the command line — demonstrating FAIR in your own institution's primary research data.

Session Aims & ILO Links

- 1 Query a REST API from the command line and explain how APIs embody the Interoperable principle
- 2 Explain PlantGenIE as an example of research infrastructure designed for interoperability
- 3 Apply a metadata sufficiency checklist to a real GEO/ENA dataset
- 4 Reconstruct a bioinformatics analysis pipeline from a methods section and identify reproducibility gaps
- 5 Commit all exercise outputs, commands, and findings to Git with structured documentation
- 6 Apply the FAIR essay marking criteria to your own preliminary paper assessment

ILO 1, 3

ILO 3

ILO 3, 13

ILO 3, 13

ILO 12

ILO 13

This is a practical session. Bring questions from Lecture 5. The essay assessment closes [date — see Canvas].

Today's Session: FAIR Applied

Each exercise maps directly to a FAIR principle — and to the essay assessment.

I — Interoperable



PlantGenIE API

Query the PlantGenIE REST API from the command line. Retrieve gene information, expression data, and BLAST results in standard formats. Understand why API design choices either enable or limit FAIR compliance.

Co-delivered with Jamie McCann

R — Reusable



GEO/ENA Metadata Assessment

Apply the metadata sufficiency checklist to a real dataset. Determine what is present, what is missing, and what the practical consequences of each gap would be for a researcher attempting reanalysis.

Independent exercise

R — Reusable



Paper Reproducibility Exercise

Reconstruct the analysis pipeline from your chosen essay paper. Build the evidence base you will use in the essay. Identify the most significant reproducibility gaps.

Independent exercise using your essay paper

Part 1

I = Interoperable: The PlantGenIE API

PlantGenIE: Research Infrastructure at Umeå

 Developed at Umeå University · plantgenie.se · github.com/plantgenie

Genome browsers

Reference genomes for *Picea abies* (Norway spruce) and *Populus tremula* (European aspen), with gene models, annotations, and synteny tools

Expression data

Integrated RNA-seq expression atlases across tissues and conditions, with co-expression network tools for identifying gene modules

BLAST and search

Sequence similarity search against conifer and aspen genomes — accessible via web interface and API

REST API

All data accessible programmatically via documented HTTP endpoints returning JSON and FASTA. This is what we use today.

Jamie McCann will now introduce PlantGenIE and explain the API design decisions that make it Interoperable.

Why APIs Embody the Interoperable Principle

A well-designed API means: any tool, any programming language, any researcher — access the same data in the same way.

11

Uses a formal, accessible, broadly applicable language

REST over HTTPS — the same protocol every web browser uses. Any tool that can make HTTP requests can query the API.

12

Uses vocabularies that follow FAIR principles

Gene entries include GO term identifiers (GO:XXXXXXX), InterPro accessions, NCBI taxonomy IDs — stable, machine-readable ontology terms, not free text.

13

Includes qualified references to other data

Gene entries cross-reference ENA accessions, Ensembl gene IDs, UniProt accessions — forming a web of connected resources.

+

Documented endpoints

The API documentation describes what each endpoint accepts and returns. A researcher can query it without contacting the authors or using a specific browser.

Compare: a database that only provides data via a browser download, with no stable URL and no documented structure, is not Interoperable — even if the data is technically available.

Exercise 1: Querying the PlantGenIE API

Login to Kebnekaise via OnDemand. Full commands in your handout.

Setup

Create lecture06 directory · git init · initial commit

1A — Gene info

Query gene information for a Norway spruce gene via curl · inspect JSON response · extract GO terms and cross-references

1B — Expression

Retrieve expression data across tissues · identify most-expressed tissues · note whether expression units are specified

1C — BLAST

Submit a BLAST query via the API (asynchronous: submit → wait → retrieve) · does spruce have homologs to your query sequence?

1D — Commit

Document findings in README · git add · git commit with structured message · note: what makes this workflow interoperable?

If plantgenie.se is unavailable: your instructor will provide example JSON responses to work with locally. The commands and learning objectives are identical.

Exercise 1: Key API Commands

Parts 1A and 1B — gene information and expression

```
$ BASE="https://api.plantgenie.se"
# 1A: Retrieve gene information
$ curl -s "${BASE}/genes/MA_10000g0010" > gene_info.json
$ cat gene_info.json
$ grep -o '"description": "[^"]*"' gene_info.json
$ grep -o '"go_terms": \[[^\]]*\]' gene_info.json
# 1B: Retrieve expression data across tissues
$ curl -s "${BASE}/expression/MA_10000g0010" > expression_data.json
$ grep -o '"tissue": "[^"]*"' expression_data.json | wc -l
```

Part 1C — BLAST via API (asynchronous)

```
$ QUERY=">query_seq\nMEEPQSDPSVEPPLSQETFSDLWKLLPENNVLSPLPSQAMDDLMLSPDDIE"
# Submit BLAST job
$ curl -s -X POST "${BASE}/blast" \
$   -H "Content-Type: application/json" \
$   -d '{"sequence\: "${QUERY}"', "database\: "picea_abies", "program\: "blastp"}' \
$   > blast_job.json
$ JOB_ID=$(grep -o '"job_id": "[^"]*"' blast_job.json | cut -d'"' -f4)
$ echo "Job ID: ${JOB_ID}" && sleep 30
$ curl -s "${BASE}/blast/results/${JOB_ID}" > blast_results.json && cat blast_results.json
$ git add . && git commit -m "lecture06: PlantGenIE API exercises complete"
```

Reflecting on what you just did:

1

The PlantGenIE API returns gene data as JSON with GO term identifiers like GO:0005488. Which FAIR principle does this most directly embody, and why?

2

You queried the API using curl from the command line. A colleague accesses the same data through the browser. Which approach is more interoperable — and why does interoperability benefit computational research?

3

The BLAST job was submitted asynchronously (submit → wait → retrieve). This is the same pattern as NCBI's API (from Lecture 3). Why do database APIs use this pattern for computationally expensive jobs?



Break — 10 minutes

Back at: _____

After the break: GEO metadata assessment • paper reproducibility • essay briefing

Part 2

R = Reusable: Metadata assessment and paper reproducibility

The Reusability Challenge: Can You Actually Reuse This Data?

Deposited data is not the same as reusable data.

Scenario: not reusable

A GEO entry has 12 RNA-seq samples. Raw FASTQ files are available. The samples are labelled "sample_01" through "sample_12". The GEO description says "expression profiling of treated vs untreated cells." There are no ontology terms, no specification of cell type, no indication of which samples are treated and which are control, and the reference genome version is not stated.

Result: a researcher who finds this dataset cannot determine experimental conditions without contacting the authors. The data exists, but it is not reusable.

Scenario: reusable

The same 12 samples, but each is annotated with: organism (NCBI:txid9606), tissue (UBERON:0002368), treatment (EFO:0001461 for dexamethasone treatment), concentration and duration, and a link to the reference genome accession (GCF_000001405.40). Each condition has 3 biological replicates clearly labelled. Analysis code is available with a Zenodo DOI.

Result: an independent researcher can immediately reproduce the differential expression analysis from the raw data alone.

The difference is not the data — it is the metadata. This is exactly what your exercise assesses.

Exercise 2: The Metadata Sufficiency Checklist

Dataset: ERP016242 · Sundell et al. (2017) AspWood · *Populus tremula* wood formation · The Plant Cell

Sample identification

- ☐ Findable by search without knowing accession in advance?
- ☐ All samples identifiable and assignable to experimental conditions?
- ☐ Biological replicates clearly labelled?

Biological metadata

- ☐ Organism with NCBI taxonomy ID?
- ☐ Tissue/cell type with UBERON or CL ontology term?
- ☐ Treatment with EFO or ChEBI ontology term?
- ☐ Other variables (age, sex, genotype, time point)?

Technical metadata

- ☐ Sequencing platform specified?
- ☐ Library type (stranded, poly A/(ribo-depl.) specified?

Exercise 3: Paper Reproducibility Assessment

Open your chosen essay paper. Work through the following systematically.

1

Map the analysis pipeline

List every computational step in the methods. For each: tool, version specified?, parameters reported? Build a table — see example below.

2

Assess code availability

Is code available at all? If yes: is it versioned (Git tag/Zenodo), documented, with dependencies pinned? If no: is there any reproducibility path?

3

Assess the reference genome

Is the reference genome specified? Is the assembly accession version given? Is the annotation GTF version specified? Could you identify exactly the right reference to reproduce the alignment?

4

Write a reproducibility verdict

2-3 sentences: could an independent researcher reproduce the main analysis? What specific information would they need from the authors that is not in the paper?

Example pipeline table in your handout. This becomes the evidence base for your essay — take careful notes.

Example: Reconstructing the Analysis Pipeline

Build this table from the methods section of your chosen paper. This is your FAIR evidence base.

Step	Tool	Version?	Parameters?	Notes
Quality trimming	Trimmomatic	v0.39 ✓	LEADING:3 TRAILING:3 MINLEN:36 ✓	Well reported
Alignment	HISAT2	v2.1.0 ✓	'default parameters' ✗	Parameters missing
Quantification	featureCounts	Not specified ✗	Not specified ✗	Version + params missing
DE analysis	DESeq2	v1.20.0 ✓	Shrinkage method not stated ✗	Partial — key parameter absent
Reference genome	GRCh38	Release not specified ✗	—	Cannot confirm exact version

'Default parameters' is almost never sufficient for reproducibility — defaults change between software versions, and many tools have multiple default modes depending on other settings. This is one of the most common reproducibility failures in bioinformatics methods sections.

Essay Briefing

From today's exercises to the Canvas assignment

From Exercise to Essay: The Direct Connection

Everything you have done today is direct preparation for the essay.

Exercise: PlantGenIE API exercise

→ Essay: Evidence for the Interoperable principle — you have now experienced what a well-designed API enables. Use this as a reference point when assessing I in your essay paper.

Exercise: GEO metadata checklist

→ Essay: Direct template for your Reusability assessment. Apply the same checklist to your essay paper's dataset. Your completed checklist from today is your methodology.

Exercise: Pipeline reconstruction table

→ Essay: This IS your reproducibility evidence. The table you built today becomes the backbone of your essay's computational reproducibility section.

Exercise: Reproducibility verdict

→ Essay: The 2-3 sentence verdict you wrote today is the draft of your essay's synthesis paragraph. Develop it with specific consequences and recommendations.

G vs VG: The Distinction That Matters

G

G — Description

""The sample metadata in the GEO entry does not include ontology terms for the treatment condition.""

Identifies the issue correctly. States what is absent.

VG

VG — Analysis + Consequence + Recommendation

""The treatment condition is described only as 'treated' with no ontology annotation. This means a researcher querying GEO for all datasets involving dexamethasone treatment (EFO:0001461) would not retrieve this dataset — it is effectively invisible to semantic database searches. A CHEBI term for the compound and an EFO term for the treatment type would have been straightforward to

Both responses describe the same fact. The VG response explains what it means — for discoverability, for reuse, and for the researcher who finds the dataset five years from now.

Identifies the issue. Explains the mechanism. Quantifies the consequence. Names the specific fix.

What You Should Know After This Session



Query a REST API from the command line and parse JSON responses with grep



Explain how APIs embody FAIR Interoperability — standard protocols, ontology terms, cross-references, documented endpoints



Describe PlantGenIE as research infrastructure designed for programmatic access to boreal tree genomics data



Apply the metadata sufficiency checklist to a GEO/ENA dataset and identify specific reusability gaps



Reconstruct a bioinformatics analysis pipeline from a methods section, noting version and parameter gaps



Write a reproducibility verdict identifying what is missing and what it would prevent



Commit all exercise outputs and findings to Git with structured README documentation



Describe the G/VG distinction: description vs analysis-of-consequence-with-recommendation